1BPLC105B – PYTHON PROGRAMMING

IPCC (Theory + Practical) | Sections M and N

Coverage: Modules 1–3 | Transactional Activities | Target Activities (GP, IP and Viva-voce)

How to use this answer key

- Use the transactional solutions for classroom demonstration, student practice and evaluation.
- For group activities, divide students into groups of four; each group should explain its logic and demonstrate at least one program.
- For individual performance, students should write the algorithm, code, and expected output without copying.
- Accept logically equivalent Python solutions. Minor formatting differences in output should not affect correctness.
- NumPy activities require: pip install numpy. All other examples use Python's standard library.

Note: The M and N section schedules contain the same activity themes, with some differences in dates. This answer key is common to both sections.

MODULE 1 – Errors, Expressions, Iteration and Functions

Transactional Activity 1 – Error Hunt

Objective: Identify and classify syntax, runtime and semantic (logical) errors.

Solution / Answer

Classifications and corrections:

```
# 1. Syntax error: missing colon
if age >= 18
    print('Eligible')

# Correct version
if age >= 18:
    print('Eligible')

# 2. Runtime error: division by zero
x = 10
y = 0
# print(x / y)  # raises ZeroDivisionError

# Safe version
if y != 0:
    print(x / y)
else:
    print('Cannot divide by zero')

# 3. Semantic/logical error: wrong formula
length = 5
breadth = 3
area = 2 * (length + breadth)  # wrong: this is perimeter

# Correct area
area = length * breadth
print('Area =', area)
```

Answers: (1) SyntaxError – Python grammar is violated. (2) ZeroDivisionError – operation is invalid at execution time. (3) Semantic error – program runs but computes the wrong result.

Expected output: Area = 15 (for the corrected area calculation).

Teaching note: Ask students to distinguish an error that prevents execution from one that produces an incorrect answer.

Transactional Activity 2 – Data Types, Operators and Expression Evaluation

Objective: Match values with data types, identify operators/converters and evaluate expressions.

Solution / Answer

Answer key for common values: 25 → int; 3.14 → float; 'Python' → str; True → bool; [1, 2] → list; (1, 2) → tuple; {'a': 1} → dict.

```
print(type(25).__name__)
print(type(3.14).__name__)
print(type('Python').__name__)

# Type conversion
print(int('25') + 5)       # 30
```

```
print(float('2.5') * 2)    # 5.0
print(str(100) + ' marks') # 100 marks

# Operator precedence
print(2 + 3 * 4)       # 14
print((2 + 3) * 4)     # 20
print(17 // 5)         # 3
print(17 % 5)          # 2
print(2 ** 3 ** 2)     # 512 (right associative)
print(10 > 5 and 3 < 4) # True
```

Expected output: 14, 20, 3, 2, 512 and True for the displayed expressions.

Transactional Activity 3 – Collatz Sequence, Nested Loops and Functions

Objective: Write and trace iterative programs and functions with arguments and return values.

Solution / Answer

```
# Collatz 3n+1 sequence
n = int(input('Enter a positive integer: '))
if n <= 0:
    print('Enter a positive integer')
else:
    while n != 1:
        print(n, end=' -> ')
        if n % 2 == 0:
            n = n // 2
        else:
            n = 3 * n + 1
    print(1)

# Multiplication tables using nested loops
for table in range(1, 4):
    print('Table of', table)
    for i in range(1, 6):
        print(table, 'x', i, '=', table * i)

# Function with argument and return value
def factorial(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

print('Factorial =', factorial(5))
```

Expected output: For input 6: 6 → 3 → 10 → 5 → 16 → 8 → 4 → 2 → 1; factorial(5) = 120.

TARGET 1 – Consolidated Assessment

A. Group Performance (groups of four)

Suggested group problem set and model answers:

- Group member 1 – Error classification: missing colon = SyntaxError; 10/0 = ZeroDivisionError; using perimeter formula for area = logical error.
- Group member 2 – Expression tracing: 5 + 2 * 3 = 11; (5 + 2) * 3 = 21; 19 // 4 = 4; 19 % 4 = 3.
- Group member 3 – Loop tracing: for i in range(1, 4): print(i*i) prints 1, 4, 9.
- Group member 4 – Function: def square(n): return n*n; square(7) returns 49.

B. Individual Performance – Practice questions with answers

- Q: What is a syntax error? A: A violation of Python grammar, such as a missing colon.
- Q: What is the output of 10 // 3? A: 3.
- Q: What is the output of 10 % 3? A: 1.
- Q: What does break do? A: Terminates the nearest enclosing loop.
- Q: What does continue do? A: Skips the remaining statements in the current iteration.
- Q: What is a function? A: A reusable named block of code; it may accept arguments and return a value.

```
def is_even(n):
    return n % 2 == 0

for value in range(1, 6):
    print(value, is_even(value))
```

Expected: 1 False; 2 True; 3 False; 4 True; 5 False.

C. Viva-voce questions

- Differentiate syntax, runtime and semantic errors.
- What is the difference between /, // and %?
- Explain operator precedence with an example.
- Differentiate for and while loops.
- Explain break and continue.
- What are parameters, arguments and return values?
- Why should a loop have a termination condition?

MODULE 2 – Strings, Tuples and Lists

Transactional Activity 1 – String Challenge

Objective: Perform slicing, find(), split() and string formatting.

Solution / Answer

```
text = 'Python Programming'
print(text[0:6])       # Python
print(text[-11:])      # Programming
print(text[::2])       # every second character
print(text.find('Pro')) # 7
print(text.split())     # ['Python', 'Programming']

name = 'Anu'
marks = 87
print('{} scored {} marks'.format(name, marks))
print(f'{name} scored {marks} marks')
```

Expected output: Python; Programming; find('Pro') returns 7; split() returns ['Python', 'Programming']; formatted sentence: Anu scored 87 marks.

Transactional Activity 2 – Tuple Assignment and Return Values

Objective: Swap variables using tuple assignment, return multiple values and compare tuples with lists.

Solution / Answer

```
a, b = 10, 20
a, b = b, a
print(a, b)  # 20 10

def calculate(a, b):
    return a + b, a * b

result = calculate(4, 5)
print(result)       # (9, 20)
sum_value, product = calculate(4, 5)
print(sum_value, product)

# Tuple vs list
my_tuple = (1, 2, 3)
my_list = [1, 2, 3]
my_list[0] = 100
print(my_list)
# my_tuple[0] = 100  # TypeError: tuple is immutable
```

Expected output: 20 10; (9, 20); 9 20; [100, 2, 3].

Transactional Activity 3 – Aliasing, Cloning, List Methods and Matrix Addition

Objective: Predict list outputs and demonstrate aliasing, cloning, list methods and matrix addition.

Solution / Answer

```
# Aliasing: both names refer to the same list
a = [10, 20, 30]
b = a
b[0] = 99
print(a)  # [99, 20, 30]
```

```
# Cloning: independent outer list
x = [10, 20, 30]
y = x[:]
y[0] = 99
print(x)  # [10, 20, 30]
print(y)  # [99, 20, 30]

numbers = [4, 2, 8]
numbers.append(5)
numbers.insert(1, 10)
numbers.remove(2)
numbers.sort()
print(numbers)  # [4, 5, 8, 10]

# Matrix addition
A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]
C = [[A[i][j] + B[i][j] for j in range(2)] for i in range(2)]
print(C)  # [[6, 8], [10, 12]]
```

Expected output: [[6, 8], [10, 12]] for matrix addition.

TARGET 2 – Consolidated Assessment

A. Group Performance – String/List/Tuple problem set

- String reverse: 'Python'[::-1] gives 'nohtyP'.
- Count vowels: sum(1 for ch in word.lower() if ch in 'aeiou').
- Tuple unpacking: x, y, z = (2, 4, 6) gives x=2, y=4, z=6.
- List aliasing: b=a means changes through b are visible through a.
- Cloning: b=a[:] creates a separate shallow copy.
- Matrix addition example: [[1,2],[3,4]] + [[5,6],[7,8]] = [[6,8],[10,12]].

B. Individual Performance – model programs

```
# 1. Count vowels in a string
text = input('Enter text: ').lower()
count = 0
for ch in text:
    if ch in 'aeiou':
        count += 1
print('Vowels =', count)

# 2. Remove duplicates while preserving order
items = [1, 2, 2, 3, 1, 4]
unique = []
for item in items:
    if item not in unique:
        unique.append(item)
print(unique)

# 3. Find maximum in a list
values = [12, 45, 7, 32]
print(max(values))
```

Expected: the duplicate-removal program prints [1, 2, 3, 4]; maximum is 45.

C. Viva-voce questions

- What is string slicing? A: Extracting a substring using start:stop:step.

- Why are strings immutable? A: Their characters cannot be modified in place.
- What is the difference between find() and index()? A: find() returns -1 when absent; index() raises ValueError.
- What is tuple packing and unpacking? A: Grouping values into a tuple and assigning tuple elements to variables.
- What is aliasing? A: Multiple references point to the same object.
- Differentiate append() and extend(). A: append adds one object; extend adds elements from an iterable.
- What is a nested list? A: A list containing other lists, often used to represent matrices.

MODULE 3 – Dictionaries, NumPy and Files

Transactional Activity 1 – Word-Frequency Counter

Objective: Count the frequency of each word using a dictionary.

Solution / Answer

```
text = input('Enter a sentence: ').lower()
words = text.split()
frequency = {}
for word in words:
    word = word.strip('.,!?;:')
    if word:
        frequency[word] = frequency.get(word, 0) + 1
print(frequency)

# Example without input
sample = 'python is easy and python is powerful'
counts = {}
for word in sample.split():
    counts[word] = counts.get(word, 0) + 1
print(counts)
```

Expected output: {'python': 2, 'is': 2, 'easy': 1, 'and': 1, 'powerful': 1}.

Transactional Activity 2 – NumPy Shape, Slicing, Masking and Broadcasting

Objective: Predict array shapes and outputs; use slicing, Boolean masks, broadcasting and dtype.

Solution / Answer

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])
print(arr.shape)       # (2, 3)
print(arr.ndim)        # 2
print(arr.dtype)       # integer dtype
print(arr[:, 1])       # [2 5]
print(arr[arr > 3])    # [4 5 6]

# Broadcasting: add a vector to each row
row = np.array([10, 20, 30])
print(arr + row)

# Explicit dtype
values = np.array([1, 2, 3], dtype=np.float32)
print(values.dtype)
```

Expected output: Shape is (2, 3); second column is [2 5]; masked values are [4 5 6]; broadcast result is [[11 22 33], [14 25 36]].

Teaching note: Broadcasting works when dimensions are equal or one of them is 1.

Transactional Activity 3 – Text File Write, Read and Sort

Objective: Write lines to a text file, read them line-by-line and sort the contents.

Solution / Answer

```
# Write data to a text file
names = ['Ravi', 'Anu', 'Kiran', 'Bala']
```

```
with open('names.txt', 'w', encoding='utf-8') as file:
    for name in names:
        file.write(name + '\n')

# Read line-by-line
with open('names.txt', 'r', encoding='utf-8') as file:
    read_names = [line.strip() for line in file]

# Sort and display
read_names.sort()
print(read_names)

# Write sorted values to another file
with open('sorted_names.txt', 'w', encoding='utf-8') as file:
    file.write('\n'.join(read_names))
```

Expected output: ['Anu', 'Bala', 'Kiran', 'Ravi'].

Teaching note: Use with open(...) so that the file is closed automatically. The file is created in the program's working directory.

TARGET 3 – Consolidated Assessment

A. Group Performance – Dictionary/NumPy/File problem set

- Dictionary access: d['name'] retrieves a value; d.get('age', 0) safely supplies a default.
- Frequency update: d[word] = d.get(word, 0) + 1.
- NumPy shape of np.array([[1,2,3],[4,5,6]]) is (2,3).
- For arr=np.array([1,2,3]), arr*2 gives [2,4,6].
- For arr=np.array([[1,2],[3,4]]), arr[:,0] gives [1,3].
- File modes: r=read, w=write/overwrite, a=append, rb=binary read, wb=binary write.

B. Individual Performance – model programs

```
# 1. Student marks dictionary
marks = {'Asha': 85, 'Ravi': 72, 'Meena': 91}
name = 'Meena'
print(marks.get(name, 'Not found'))

# 2. NumPy: row-wise total
import numpy as np
scores = np.array([[10, 20, 30], [15, 25, 35]])
print(scores.sum(axis=1))  # [60 75]

# 3. Count lines in a file
with open('sample.txt', 'r', encoding='utf-8') as f:
    lines = f.readlines()
print('Line count =', len(lines))
```

Expected: 91; [60 75]; line count depends on the contents of sample.txt.

C. Viva-voce questions

- What is a dictionary? A: A mutable mapping of unique keys to values.
- Can dictionary keys be lists? A: No; keys must be hashable.
- What does get() do? A: Returns the value for a key or a default if absent.
- What is NumPy? A: A Python library for efficient multidimensional arrays and numerical operations.
- What does shape return? A: The size along each array dimension.

- Explain broadcasting. A: NumPy's mechanism for applying operations to compatible shapes.
- Differentiate text and binary files. A: Text files store character data; binary files store bytes.
- Why use a context manager for files? A: It handles file closure reliably.